

NodeJS / JavaScript: Bundler, Framework

jQuery Prototype MooTools Dojo YUI Backbone
Knockout **Ember** AngularJS Angular **React**
Vue Meteor Aurelia **Polymer** Mithril Riot Marko
Svelte Preact Inferno Hyperapp Cycle.js Stimulus
Alpine.js Lit **Solid** Qwik **Astro** **Next.js** Nuxt
Gatsby **Remix** SvelteKit RedwoodJS Blitz Ionic Sencha
Ractive Vaadin **Om** Choo Ampersand Sammy.js

Same thing is happening to Agent Harness

Harnesses utilize the Language Server Protocol (LSP) to gain deep language intelligence, such as real-time AST parsing and diagnostics, and the Model Context Protocol (MCP) to dynamically connect to, and local tools.

They rely on runtime isolation layers like MicroVMs and container sandboxes (e.g., Docker Sandboxes). This creates a hard security boundary that allows agents to perform unattended code execution, without risking the host machine.

Sandbox isolation layers: Dedicated, isolated, Environments like Docker Sandboxes that mount specific workspaces into dedicated microVMs.

Strict perimeters established to allow processes to execute code safely and unattended without risking the broader host system.

Environments: Secure microVMs or containers provisioned to keep operations completely separated from other systems.

Capabilities: Sandboxing features that allow isolated environments to spin up their own nested, firewall-restricted containers.

ers, organizations, projects, policies, secrets, memory, artifacts, and compute.

security boundaries: A term used to describe the isolated, instance-specific permission policies (such as those in a permissions.yaml file) that dictate exactly what a specific tenant or agent is

To bridge communication between the CLI and the editor UI without hanging the terminal, harnesses tap into Inter-Process Communication (IPC) sockets like VSCODE_IPC_HOOK_CLI or utilize "wait" to capture user input or authentication credentials natively.

l of loading every instruction globally, harnesses use directory-specific gateways (such as .cursor/rules/*.mdc or .github/instructions/) to trigger specialized contextual prompts only when an agent

ce: Autonomous harnesses heavily prioritize state tracking to prevent context window exhaustion and "lost in the middle" recall issues. They manage this through rolling summaries, chronological ll), and persistent state files (state.json) that track active directives across sessions.

developers must intentionally design the harness to balance deterministic, script-based workflows with dynamic LLM-driven autonomous routing (like the ReAct or Plan-and-Execute patterns) to prevent dead-end paths or getting stuck in infinite loops in production.

rappers around shell, filesystem, Git, HTTP, browser, databases, cloud APIs, etc. instead of unrestricted raw execution.

cover available capabilities progressively rather than loading every tool/schema into context.

y / ask / scoped policies per tool, command, path, user, agent, repository, or environment.

nts temporary, restricted capabilities rather than inheriting the parent agent's full authority.

posed shell/tool operations before execution; blocks dangerous patterns or rewrites them into safer operations.

mits tokens, model spend, execution time, disk, memory, processes, API calls, network traffic, and concurrency.

main allowlists, egress controls, proxying, DNS restrictions, credential-aware routing, and offline execution.

Worktrees, overlays, temporary directories, snapshots, read-only mounts, scoped writable paths, and rollback.

t: Creates reproducible runtime environments and injects only the dependencies/configuration required for a task.

ects credentials just-in-time without putting secrets directly into prompts or agent-visible state.

etermines what a user, agent, subagent, skill, or workflow is authorized to perform.

rganizational rules independently of the LLM: compliance, security, data handling, deployment, licensing, etc.

e → inspect → approve/reject/edit → resume, with escalation based on risk rather than every operation.

execution state so a task can resume after crashes, restarts, model failures, or human intervention.

s from checkpoints, tries alternatives, compares results, or rewinds to a known-good state.

a 3-hour agent run into a recoverable workflow instead of one giant fragile LLM conversation.

t Defines reverse operations when an agent partially completes a multi-step transaction.

ats generated code, plans, reports, patches, screenshots, logs, test results, etc. as first-class versioned artifacts.

cts what goes into the model context instead of blindly appending conversation history.

mmarizes, deduplicates, ranks, chunks, expires, and retrieves information according to relevance.

xposes capability contracts first and implementation details only when required.

uilds structural maps of repositories: languages, packages, dependencies, ownership, symbols, tests, build systems, etc.

ver code, documentation, tickets, architecture decisions, prior executions, and organizational knowledge.

ple operations to cheap/fast models and difficult reasoning to stronger models.

work to specialized agents such as planner, coder, reviewer, tester, security reviewer, researcher, etc.

→ skill → tools rather than requiring the model to rediscover procedures every run.

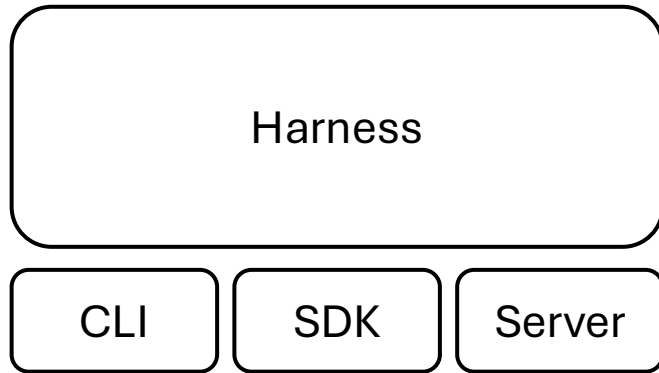
verts an objective into tasks, dependencies, DAGs, milestones, and executable units.

ndependent agents/tasks concurrently while controlling shared state and merge conflicts.

boxes, queues, blackboards, event buses, structured messages, ownership, locks, and synchronization.

Evaluation: Ru

AuditabilityMa



Idempotency: An idempotent script yields the exact same result whether executed once or a thousand times

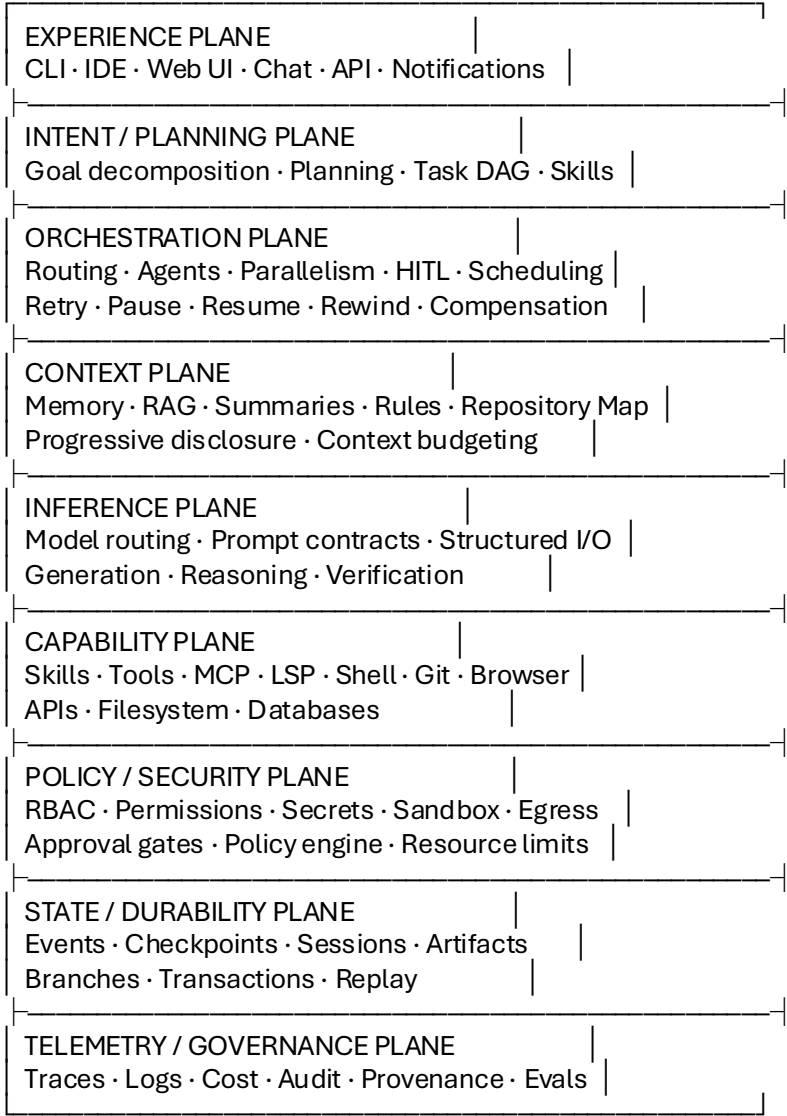
Detection (State Check): Before modifying anything, the system must query the environment to assess if the required tool or its dependencies already exist.

Resolution (Version Matching): The system must parse the currently installed version and compare it against required constraints. If the version matches, the setup skips directly to verification; if it is outdated, it moves to provisioning.

Provisioning (Fetching and Installing): If the required tool is missing or outdated, the system acquires it by handing off the request to a system package manager, fetching a direct binary from a remote source, or unpacking and symlinking to the user's path.

Configuration (Environment Hooking): Once on the disk, the software must be hooked into the user's environment by injecting paths into files like `.bashrc` or `.zshrc`, or by generating default settings files.

AGENT HARNESS



The harness shouldn't make the LLM responsible for capabilities that software can enforce deterministically.

LLM responsibility

- Understand intent
- Reason
- Plan
- Choose capabilities
- Diagnose failures
- Generate alternatives
- Make fuzzy judgments

Harness responsibility

- Permissions
- State
- Schemas
- Retries
- Timeouts
- Budgets
- Transactions
- Isolation
- Secrets
- Logging
- Audit
- Validation
- Execution
- Recovery

The harness shouldn't make the LLM responsible for capabilities that software can enforce deterministically.



This also explains why **MCP, LSP, Docker, state.json, rules files, and ReAct** aren't themselves the harness. They're mechanisms plugged into different planes.

The harness is the layer that determines **what gets exposed, when it gets exposed, who can invoke it, under what policy, with what state, how execution is verified, and how the system recovers when something fails.**

That is the difference between an **agent that can use tools** and an **agent runtime capable of safely operating software autonomously.**